

TRABAJO FINAL DE SEMESTRE

Aplicación Web Full Stack

React · Node.js · Express · MySQL

React Vite + TailwindCSS	Node.js Express REST API	MySQL Sequelize / mysql2	Seguridad bcrypt + JWT
------------------------------------	------------------------------------	------------------------------------	----------------------------------

Descripción General

¿Qué es este trabajo y qué competencias desarrolla?

Presentación

El presente trabajo consiste en el diseño, modelado e implementación de una aplicación web completa de tipo cliente/servidor. El equipo de trabajo recibirá un enunciado de dominio específico (sistema de gestión, inventario, reservas, etc.) que deberá modelar, diseñar y desarrollar aplicando los conceptos vistos durante el semestre.

La aplicación deberá seguir la arquitectura de tres capas establecida por el docente, utilizando las tecnologías del stack moderno JavaScript: React con Vite en el frontend, Node.js con Express en el backend y MySQL como base de datos relacional.

Competencias que se evalúan

Competencia	Descripción
Modelado de datos	Construir un Modelo Entidad-Relación correcto a partir de un enunciado de dominio.
Arquitectura REST	Diseñar e implementar endpoints RESTful con los verbos HTTP correctos (GET, POST, PUT, DELETE).
Autenticación segura	Implementar login con hash de contraseñas (bcrypt) y control de sesión.
Desarrollo frontend	Construir interfaces reactivas con React, Vite y TailwindCSS.
Integración de capas	Conectar correctamente frontend ↔ backend ↔ base de datos con Axios.
Control de versiones	Gestionar el proyecto en Git con commits descriptivos e instrucciones de instalación.
Documentación técnica	Redactar documentación clara de la arquitectura y el modelo de datos.
Trabajo en equipo	Distribuir responsabilidades y sustentar individualmente cualquier parte del sistema.

Conformación de grupos

- Grupos de 2 a 3 estudiantes, asignados por el profesor.
- Cada grupo recibe un enunciado de dominio diferente del docente.
- No se permite repetir el mismo enunciado entre grupos.
- Los roles (frontend, backend, base de datos) deben distribuirse equitativamente.



Importante

Cualquier evidencia de copia entre grupos resultará en calificación de 0.0 para todos los integrantes involucrados. El uso de IA como herramienta de apoyo está permitido y debe documentarse; la copia directa sin comprensión no.

Arquitectura de tres capas

La aplicación sigue el patrón Cliente/Servidor con una separación clara de responsabilidades entre tres capas. Esta arquitectura es el estándar de la industria para aplicaciones web empresariales modernas.



Frontend — Capa de Presentación [React + Vite + TailwindCSS + Axios]

Responsable de la interfaz de usuario. Captura entradas del usuario, envía peticiones HTTP al backend mediante Axios y actualiza la UI con las respuestas. No accede directamente a la base de datos.

↕ HTTP (JSON) via Axios ↕



Backend — Capa de Lógica de Negocio [Node.js + Express + bcrypt + JWT]

Actúa como intermediario. Define las rutas API RESTful, valida datos, aplica reglas de negocio, gestiona la autenticación y se comunica con la base de datos. Nunca expone credenciales de BD al frontend.

↕ SQL queries via mysql2 / Sequelize ↕



Base de Datos — Capa de Persistencia [MySQL + Modelo relacional normalizado]

Almacena todos los datos del dominio en tablas estructuradas. El diseño debe estar normalizado al menos en 3FN. El backend se conecta usando el driver mysql2 o el ORM Sequelize.

Flujo de comunicación

{ } Flujo típico de una petición (ejemplo: listar registros)

1. Usuario hace clic en 'Ver lista' en el frontend React
2. React ejecuta: `axios.get('http://localhost:3001/api/entidades')`
3. El request llega al servidor Express
4. Express valida el JWT del header Authorization
5. El controlador ejecuta: `SELECT * FROM entidades` (via mysql2/Sequelize)
6. MySQL devuelve los registros al controlador
7. Express responde: `res.json({ data: registros })` — HTTP 200
8. Axios recibe la respuesta y React actualiza el estado con `setData(response.data)`
9. React re-renderiza la tabla con los nuevos datos

Puertos y configuración estándar

Componente	URL / Configuración
Frontend (React/Vite)	http://localhost:5173
Backend (Node.js/Express)	http://localhost:3001
Base de Datos (MySQL)	localhost:3306 — BD: nombre_proyecto_db
Variable de entorno JWT	JWT_SECRET en archivo .env (nunca en el código)
Variable de entorno BD	DB_HOST, DB_USER, DB_PASSWORD, DB_NAME en .env



Recomendación

Usar el archivo .env para todas las credenciales y secretos. Agregar .env al .gitignore para que no suba al repositorio. Incluir un archivo .env.example con los nombres de las variables pero sin los valores reales.

RF-01 — Autenticación de Usuarios (NOVEDAD DEL CURSO)

El sistema debe implementar un módulo completo de autenticación de usuarios. Es la puerta de entrada a toda la aplicación.

Tabla de usuarios en MySQL (obligatoria)

{ } SQL — Tabla usuarios	
CREATE TABLE usuarios (	
id	INT AUTO_INCREMENT PRIMARY KEY,
usuario	VARCHAR(50) NOT NULL UNIQUE,
password	VARCHAR(255) NOT NULL, -- hash bcrypt (mínimo 10 rounds)
imagen	VARCHAR(255) DEFAULT NULL, -- ruta o URL de la imagen de perfil
rol	ENUM('admin','usuario','moderador') NOT NULL DEFAULT 'usuario',
creado_en	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);	

Requisitos de seguridad del login

- Las contraseñas deben almacenarse con hash bcrypt (mínimo 10 rounds). NUNCA en texto plano.
- El backend debe generar un JWT (JSON Web Token) al login exitoso con expiración de 8 horas.
- Cada petición al backend debe incluir el JWT en el header: Authorization: Bearer <token>.
- El frontend debe guardar el token en localStorage o sessionStorage y enviarlo en cada request.
- Las rutas protegidas deben verificar el JWT antes de procesar cualquier solicitud.
- Implementar cierre de sesión (logout) que elimine el token del cliente.

{ } Node.js — Ejemplo de registro y login con bcrypt + JWT	
const bcrypt = require('bcrypt');	
const jwt = require('jsonwebtoken');	
// REGISTRO: hashear la contraseña antes de guardar	
const ROUNDS = 10;	
const hash = await bcrypt.hash(req.body.password, ROUNDS);	
await db.query('INSERT INTO usuarios (usuario, password, rol) VALUES (?, ?, ?)',	
[req.body.usuario, hash, 'usuario']);	
// LOGIN: verificar la contraseña	
const [rows] = await db.query('SELECT * FROM usuarios WHERE usuario = ?',	
[req.body.usuario]);	
const esValido = await bcrypt.compare(req.body.password, rows[0].password);	
if (esValido) {	
const token = jwt.sign(	
{ id: rows[0].id, rol: rows[0].rol },	
process.env.JWT_SECRET,	
{ expiresIn: '8h' }	
);	
res.json({ token, usuario: rows[0].usuario, rol: rows[0].rol });	
}	

RF-02 — Dashboard Principal

Después del login, el usuario debe llegar a un Dashboard que sirva como hub de navegación. El dashboard debe mostrar:

- Nombre de usuario y foto de perfil (o avatar por defecto si no hay imagen).
- Rol del usuario (admin, usuario, moderador) con indicador visual.

- Menú de navegación con todas las opciones disponibles según el rol.
- Al menos 3 tarjetas de resumen (cards) con estadísticas del dominio (ej: total de registros, últimas operaciones).
- Botón de cierre de sesión visible en todo momento.

i Nota
El diseño del dashboard debe construirse con TailwindCSS. Se pueden usar librerías de íconos como Heroicons, Lucide-React o React-Icons para enriquecer la UI.

RF-03 — Módulo CRUD del Dominio

Según el enunciado recibido, el sistema debe implementar al menos un módulo CRUD completo para la entidad principal del dominio. Cada operación tiene requisitos específicos:

Operación	Requisito
Listar (GET)	Tabla paginada con búsqueda y filtros. Mostrar todos los campos relevantes.
Crear (POST)	Formulario controlado React con validación en frontend y backend. Confirmación SweetAlert2.
Editar (PUT/PATCH)	Formulario pre-cargado con los datos actuales. Confirmación de cambios con SweetAlert2.
Eliminar (DELETE)	Confirmación SweetAlert2 con pregunta '¿Estás seguro?' antes de ejecutar el DELETE.

RF-04 — Ventanas de Confirmación con SweetAlert2

Todas las operaciones que modifiquen o eliminen datos deben usar SweetAlert2 para las confirmaciones. Está prohibido usar el alert() nativo del navegador o window.confirm().

{ } React — Ejemplo de SweetAlert2 para eliminación

```
import Swal from 'sweetalert2';

const confirmarEliminar = async (id, nombre) => {
  const resultado = await Swal.fire({
    title: '¿Eliminar registro?',
    text: `¿Seguro que deseas eliminar a ${nombre}? Esta acción no se puede deshacer.`,
    icon: 'warning',
    showCancelButton: true,
    confirmButtonColor: '#d33',
    cancelButtonColor: '#3085d6',
    confirmButtonText: 'Sí, eliminar',
    cancelButtonText: 'Cancelar',
  });
};

if (resultado.isConfirmed) {
  try {
    await axios.delete(`/api/entidades/${id}`,
      { headers: { Authorization: `Bearer ${token}` } }
    );
    Swal.fire('Eliminado', 'El registro fue eliminado correctamente.', 'success');
    cargarDatos(); // Refrescar la lista
  } catch (error) {
    Swal.fire('Error', 'No se pudo eliminar el registro.', 'error');
  }
}
};
```

RF-05 — Control de Acceso por Rol

El sistema debe implementar control de acceso basado en roles (RBAC). Como mínimo, definir dos niveles de acceso:

- Rol admin: acceso completo a todas las opciones, incluyendo gestión de usuarios.
- Rol usuario: acceso limitado a las operaciones de su propio dominio.
- Las rutas del backend deben verificar el rol antes de ejecutar operaciones sensibles.
- El frontend debe ocultar o deshabilitar los elementos de UI que el usuario no tiene permiso para usar.

Estructura de carpetas obligatoria

{ } Estructura del repositorio Git	
nombre-proyecto/	
├── frontend/	← Proyecto React + Vite
│ ├── src/	
│ │ ├── components/	← Componentes reutilizables
│ │ ├── pages/	← Páginas (Login, Dashboard, módulos)
│ │ ├── hooks/	← Custom hooks (useAuth, useFetch...)
│ │ ├── context/	← Context API para estado global
│ │ ├── services/	← Funciones Axios (api.js, authService.js)
│ │ └── App.jsx	← Rutas con React Router
│ ├── .env	← Variables de entorno frontend
│ ├── tailwind.config.js	
│ └── package.json	
├── backend/	← Servidor Node.js + Express
│ ├── controllers/	← Lógica de cada ruta
│ ├── routes/	← Definición de endpoints API
│ ├── middleware/	← verifyToken.js, checkRole.js
│ ├── models/	← Modelos Sequelize (o consultas mysql2)
│ ├── config/	
│ │ └── db.js	← Conexión a MySQL
│ ├── .env	← Credenciales y JWT_SECRET
│ ├── .env.example	← Plantilla sin valores reales
│ ├── server.js	← Punto de entrada
│ └── package.json	
├── database/	
│ ├── schema.sql	← Script de creación de tablas
│ ├── seeds.sql	← Datos de prueba
│ └── er_diagram.png	← Imagen del diagrama ER
├── docs/	
│ ├── arquitectura.pdf	← Documento de arquitectura
│ └── manual_usuario.pdf	← Guía de uso básica
└── README.md	← Instrucciones completas de instalación

Stack tecnológico — versiones mínimas

Tecnología / Librería	Versión requerida
Node.js	v18.x o superior (LTS recomendado)
React	v18.x con Vite como bundler
TailwindCSS	v3.x — instalado como plugin de PostCSS
Axios	v1.x — para peticiones HTTP desde React
React Router	v6.x — para navegación en el frontend
Express	v4.x — framework web del backend
mysql2	v3.x — driver para MySQL (o Sequelize v6)
bcrypt	v5.x — para hash de contraseñas
jsonwebtoken	v9.x — para generación y verificación de JWT
SweetAlert2	v11.x — para alertas y confirmaciones
MySQL	v8.x — base de datos relacional

cors	Último — habilitar CORS en Express
dotenv	Último — variables de entorno en Node.js

Estándares de código

- Usar variables de entorno (.env) para credenciales — nunca hardcodear.
- El backend debe responder siempre con JSON estructurado: { success, data, message }.
- Implementar manejo de errores en todos los endpoints con try/catch y códigos HTTP correctos.
- Los componentes React deben ser funcionales (sin clases) y usar hooks.
- Usar async/await en lugar de .then()/.catch() para legibilidad.
- Comentar el código en las secciones no obvias — en español o inglés, de forma consistente.
- Usar nombres descriptivos para variables, funciones y endpoints (no x, var1, cosa).

{ } Backend — Estructura estándar de respuesta JSON
// Respuesta exitosa
res.status(200).json({
success: true,
data: registros,
message: 'Registros obtenidos correctamente'
});
// Respuesta de error
res.status(500).json({
success: false,
data: null,
message: 'Error al consultar la base de datos'
});
// Ejemplo de endpoint completo con manejo de errores
router.get('/entidades', verifyToken, async (req, res) => {
try {
const [rows] = await db.query('SELECT * FROM entidades');
res.status(200).json({ success: true, data: rows });
} catch (error) {
console.error(error);
res.status(500).json({ success: false, message: 'Error del servidor' });
}
});

SEC.

5

Entregables

Qué se debe entregar, cuándo y cómo

1

Repositorio Git con README completo

Repositorio público en GitHub o GitLab con el código fuente completo. El README.md debe contener instrucciones paso a paso para instalar y ejecutar el proyecto en un equipo limpio. Incluir: requisitos previos, comandos de instalación, configuración del .env y cómo ejecutar las migraciones.

20%

2

Documentación de Arquitectura

Documento PDF o Word que describa: (1) Diagrama de arquitectura del sistema con las tres capas. (2) Descripción de cada componente y su responsabilidad. (3) Lista de endpoints de la API con método HTTP, URL, body y respuesta esperada. (4) Justificación de las decisiones técnicas tomadas.

15%

3

Modelo Entidad-Relación

Diagrama ER completo del dominio asignado. Debe incluir todas las entidades, atributos (con tipos de dato), llaves primarias y foráneas, y las relaciones entre entidades con su cardinalidad. Herramientas sugeridas: draw.io, MySQL Workbench, dbdiagram.io, Lucidchart.

15%

4

Código Fuente Funcional

El sistema completo debe funcionar sin errores. Se evaluará: login con bcrypt y JWT, dashboard, módulo CRUD del dominio, confirmaciones SweetAlert2, control de roles, código estructurado según las carpetas establecidas y uso correcto de los estándares definidos.

30%

5

Sustentación Oral

Cada integrante del grupo debe sustentar individualmente. El docente podrá hacer preguntas sobre cualquier parte del sistema. Se evaluará la comprensión del código, capacidad para explicar decisiones de diseño y aptitud para realizar modificaciones en vivo sobre el código.

20%

Cronograma de entrega

Entregable	Semana / Fecha	Modalidad de entrega
Modelo ER (borrador)	28/04/2026	Correo al docente (PDF / imagen)
Modelo ER (definitivo)	30/04/2026	Commit en repositorio Git
Avance funcional (100%)	05/05/2026	Demo en clase + enlace al repo
Código fuente completo	05/05/2026	Commit final en repositorio Git
Documentación de arquitectura	05/05/2026	PDF en carpeta /docs del repo
Sustentación oral	05/05/2026	Presencial en clase

!

Importante

No se aceptarán entregas tardías sin justificación previa. El código debe funcionar en el equipo del docente al momento de la sustentación. Se descontarán puntos si el sistema no puede ejecutarse con las instrucciones del README.

Distribución de puntos

Criterio	Indicador	Exc	Suf	Ins
Modelo Entidad-Relación	Entidades correctas, atributos con tipo, llaves, relaciones y cardinalidad correctas	5.0	3.5	< 3.0
Login con bcrypt + JWT	Hash de password, generación de token, verificación en rutas protegidas	5.0	3.5	< 3.0
Dashboard funcional	UI completa, tarjetas de resumen, foto de perfil, navegación por rol	5.0	3.5	< 3.0
CRUD del dominio	Las 4 operaciones (C-R-U-D) funcionan correctamente e integran el backend	5.0	3.5	< 3.0
SweetAlert2 en confirmaciones	Todas las confirmaciones usan SweetAlert2 con el flujo correcto	5.0	3.5	< 3.0
Control de roles	Restricciones por rol en frontend y backend verificadas	5.0	3.5	< 3.0
Documentación y README	Instrucciones completas, arquitectura documentada, ER en el repo	5.0	3.5	< 3.0
Calidad del código	Estructura de carpetas, estándares, variables de entorno, comentarios	5.0	3.5	< 3.0
Git y control de versiones	Historial de commits descriptivos, .env.example, .gitignore correcto	5.0	3.5	< 3.0
Sustentación individual	Capacidad de explicar, modificar y responder preguntas del docente	5.0	3.5	< 3.0

Exc = Excelente (cumple y supera) · Suf = Suficiente (cumple lo mínimo) · Ins = Insuficiente

Herramientas de prototipado UI (permitidas)

Para el diseño de las interfaces gráficas antes de codificar, se pueden usar las siguientes herramientas. Esto ayuda a planificar la UI antes de escribir código TailwindCSS.

Herramienta	Descripción
Pencil Project	Herramienta de wireframing de escritorio. Gratuita y de código abierto.
Google Stitch	Generador de UI con IA de Google. Disponible en stitch.google.com .
Figma	Diseño colaborativo online. Plan gratuito disponible.
draw.io / diagrams.net	Para diagramas de arquitectura y ER. Exporta a PNG/SVG.
dbdiagram.io	Diseño de diagramas ER con sintaxis simple. Gratuito online.
MySQL Workbench	IDE oficial de MySQL. Permite diseñar el ER visualmente y exportar SQL.

Guía de inicio rápido del proyecto

{ } Terminal — Crear el proyecto completo desde cero
1. Crear carpeta del proyecto
<code>mkdir nombre-proyecto && cd nombre-proyecto</code>
<code>git init</code>
2. Crear frontend con Vite + React
<code>npm create vite@latest frontend -- --template react</code>
<code>cd frontend</code>
<code>npm install</code>
<code>npm install -D tailwindcss postcss autoprefixer</code>
<code>npm run tailwindcss init -p</code>
<code>npm install axios react-router-dom sweetalert2</code>
<code>cd ..</code>
3. Crear backend Node.js + Express
<code>mkdir backend && cd backend</code>
<code>npm init -y</code>
<code>npm install express mysql2 bcrypt jsonwebtoken cors dotenv</code>
<code>npm install -D nodemon</code>
<code>cd ..</code>
4. Estructura de carpetas del backend
<code>cd backend</code>
<code>mkdir controllers routes middleware config models</code>
5. Primer commit
<code>cd ..</code>
<code>echo 'node_modules\n.env' > .gitignore</code>
<code>git add .</code>
<code>git commit -m 'feat: estructura inicial del proyecto'</code>

Recursos de estudio recomendados

Recurso	URL / Descripción
React docs	react.dev — Documentación oficial de React 18
TailwindCSS	tailwindcss.com/docs — Referencia de clases

Express.js	expressjs.com/en/guide — Guías oficiales
SweetAlert2	sweetalert2.github.io — Ejemplos y configuración
JWT.io	jwt.io — Decodificador y explicación de JWT
bcrypt npm	npmjs.com/package/bcrypt — Documentación y ejemplos
Vite	vitejs.dev/guide — Configuración de Vite

Uso de Inteligencia Artificial

El uso de herramientas de IA (ChatGPT, Claude, GitHub Copilot, Google Gemini) está PERMITIDO con las siguientes condiciones:

- Cada integrante debe comprender todo el código del proyecto, sin excepción.
- Durante la sustentación, el docente puede pedir que se explique o modifique cualquier línea de código.
- Usar IA para diseño de UI (Stitch, Pencil) está completamente permitido.
- No se acepta que el grupo no pueda explicar decisiones de diseño o fragmentos del código.



Recomendación

Una buena práctica es usar la IA para generar un borrador, revisarlo en grupo, entenderlo completamente y luego adaptarlo al dominio del proyecto. La IA es una herramienta, no un atajo para evitar el aprendizaje.